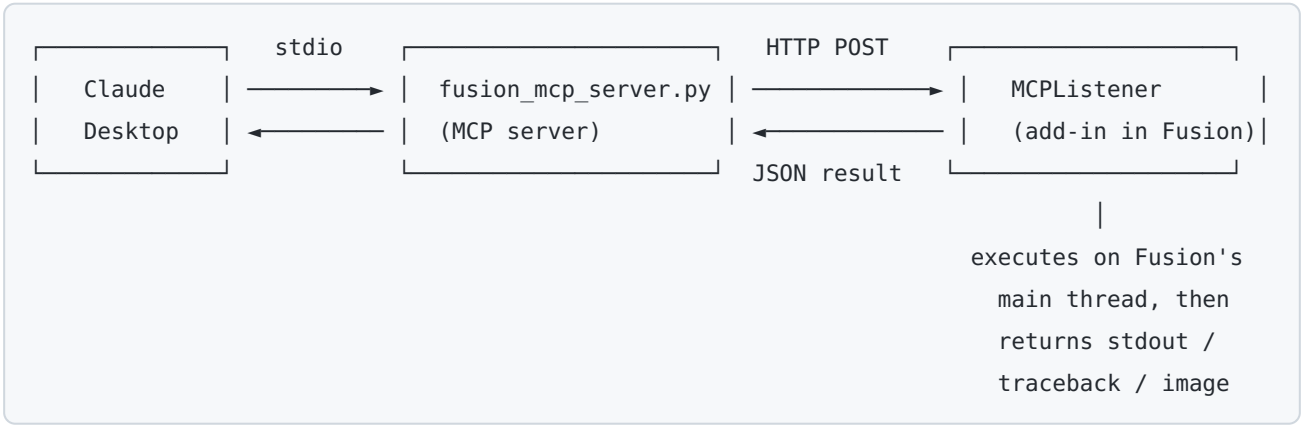
Architecture

For the curious, and for anyone wanting to extend the toolset.

The processes



Three participants:

1. **Claude Desktop** reads `claude_desktop_config.json`, launches the MCP server as a child process, and speaks the MCP protocol to it over stdio.
2. `fusion_mcp_server.py` is a FastMCP server. Each tool builds a small Python snippet (or passes user code through) and HTTP-POSTs it as JSON to the listener, returning the listener's response to Claude.
3. `MCPListener` is a Fusion add-in. It runs an `http.server` on `127.0.0.1:8765` with async endpoints: `POST /exec` (enqueue, returns job id), `GET /job/<id>` (envelope), `GET /ping` and `GET /state` (served without the main thread), and `POST /exec_sync` (bounded blocking, back-compat). Incoming code runs on the main thread; status, stdout, and tracebacks go into a job store.

Why two processes instead of one?

Fusion's Python API can only be called **from inside Fusion's own interpreter**, and only on its **main thread**. An external MCP server can't touch the API directly. So the add-in lives inside Fusion to hold API access, and the MCP server lives outside (where Claude Desktop can spawn it) and talks to the add-in over localhost HTTP.

The async job model and the main-thread handoff

An HTTP server needs a background thread to listen, but Fusion's API rejects calls from any thread except its main one. And a *hung* geometry call blocks that main thread — so no HTTP cleverness can make a queued geometry call jump ahead. The design works around this:

Jobs are async. `POST /exec` enqueues a job, fires a Fusion custom event (`MCPListenerExec`) with the job id, and returns the id **immediately**. The event is dispatched on the main thread, where a handler runs the code and writes status + stdout into a shared job store. `GET /job/<id>` returns a structured envelope (`status`, `stdout`, `traceback`, `elapsed_ms`) — and reads partial stdout while the job is still running (incremental progress via a live buffer).

Ping and state never touch the main thread. `GET /ping` and `GET /state` are answered entirely from cached data on the HTTP thread. So even while a geometry call churns (or hangs) on the main thread, liveness and cheap document state stay responsive. This is what makes "is it busy or dead?" answerable.

A watchdog flags overruns. A background thread marks any job that exceeds its budget as `status: timeout`. It can't force-kill a hung main thread (Fusion's API has no such hook), but the caller gets an immediate, unambiguous signal instead of a dead socket.

Sync path for convenience. `POST /exec_sync` enqueues then waits up to `wait_s` and returns the envelope — used by `run_fusion_code` for short ops. If the op outlives the wait, it keeps running and stays pollable by job id; the response says so and returns partial stdout. The server's HTTP client timeout sits above `wait_s`, so the agent always gets a structured envelope, never a bare socket timeout.

Units

Fusion's API is internally in **centimeters**. Every tool exposes **millimeters** to Claude and divides by 10 before talking to the API. When writing raw code via `run_fusion_code`, you're in cm — divide mm by 10 yourself.

The safety wrapper

`run_fusion_code_safe` records the timeline marker position, runs your code, and on exception rolls the timeline back and deletes any features created after the marker — leaving the document as it was. It's effective on parametric documents (design history on, the default). Direct-modeling docs have no timeline to roll back, so it degrades to plain error reporting.

Visual feedback

`view_model` orients the camera, fits, saves a PNG via `Viewport.saveAsImageFile`, reads the bytes back, and returns them as an MCP `Image`. Because the MCP server and Fusion are on the same machine, the file round-trips through `~/fusion_view.png`. (This is the one tool that assumes shared local disk.)

Security model

- The listener binds to **`127.0.0.1` only** — not reachable off-machine.
- It does, by design, **execute arbitrary Python** it receives. That's the whole mechanism. The trust boundary is: only your local MCP server (driven by your Claude Desktop) talks to it.
- **Do not** change the bind address to `0.0.0.0` or forward the port. Doing so would let anything on your network run code in your Fusion session.
- There's no authentication on the localhost channel; it relies on the OS loopback boundary. Fine for a personal machine, not for a shared/multi-user host.

Extending the toolset

Add a new tool by writing a function in `fusion_mcp_server.py` decorated with `@mcp.tool()`. Build a Fusion API snippet as a string, `print()` whatever you want Claude to see, and return `_send(code)`. Patterns to copy:

- **Take mm, convert to cm** at the top (`x = x_mm / 10.0`).
- **`print()` a clear success line** — that string is what Claude reads back.
- **Guard lookups** (component/body not found) and print an `ERROR: ...` line rather than throwing, so Claude gets a readable message.
- For multi-step features, consider routing through the same rollback idea as `run_fusion_code_safe`.

Restart Claude Desktop after editing the server so it reloads the tool list.

Files

File	Runs where	Role
`MCPListener.py`	Inside Fusion	HTTP listener + main-thread executor
`fusion_mcp_server.py`	Spawned by Claude Desktop	MCP tools → HTTP relay
`launch.py`	Your terminal	Preflight/health checks only